



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Основи проектування розгортання»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Основи проектування розгортання.

Мета: Навчитися проектувати діаграми розгортання та компонентів для системи що проектується, а також розробляти діаграми взаємодії, а саме діаграми послідовностей, на основі сценаріїв зроблених в попередній лабораторній роботі.

Тема проєкту: Mind-mapping software

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	3
Хід роботи	4
Діаграма розгортання системи.....	5
Діаграма компонентів	6
Діаграми послідовностей.....	7
Код програми	8
Висновок	22
Питання до лабораторної роботи	22

Теоретичні відомості

Діаграми розгортання представляють фізичне розташування системи, показуючи, на якому фізичному обладнанні запускається та чи інша складова програмного забезпечення.

Вузол (node) – це те, що може містити програмне забезпечення. Вузли бувають двох типів. Пристрій (device) – це фізичне обладнання: комп'ютер або пристрій, пов'язаний із системою. Середовище виконання (execution environment) – це програмне забезпечення, яке саме може включати інше програмне забезпечення, наприклад операційну систему або процес-контейнер (наприклад, вебсервер).

Між вузлами можуть стояти зв'язки, які зазвичай зображують у вигляді прямої лінії. Як і на інших діаграмах, у зв'язків можуть бути атрибути множинності (для показання, наприклад, підключення 2х і більше клієнтів до одного сервера) і назва. У назві, як правило, міститься спосіб зв'язку між двома вузлами – це може бути назва протоколу (HTTP, IPC) або технологія, що використовується для забезпечення взаємодії вузлів (.NET Remoting, WCF).

Діаграма компонентів UML є представленням проєктованої системи, розбитої на окремі модулі [3]. Залежно від способу поділу на модулі розрізняють три види діаграм компонентів:

- логічні;
- фізичні;
- виконувані.

Діаграма складається з таких основних елементів:

- Актори
- Об'єкти або класи
- Повідомлення
- Активності
- Контрольні структури

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Проаналізувати діаграми створені в попередній лабораторній роботі а також тему системи та спроектувати діаграму розгортання використання відповідно до обраної теми лабораторного циклу.
- Розробити діаграму компонентів для проєктованої системи.
- Розробити діаграму розгортання для проєктованої системи.
- Розробити як мінімум дві діаграми послідовностей для сценаріїв прописаних в попередній лабораторній роботі.
- На основі спроектованих діаграм розгортання та компонентів доопрацювати програмну частину системи. Реалізація системи, додатково до попередньої реалізації, повинна містити як мінімум дві візуальні форми. В системі вже повинен бути повністю реалізована архітектура (повний цикл роботи з даними від вводу на формі до збереження їх в БД і подальшій виборці з БД та відображенням на UI).
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму розгортання з описом, діаграму компонентів системи з описом, діаграми послідовностей, а також вихідний код системи, який було додано в цій лабораторній роботі.

Діаграма розгортання системи

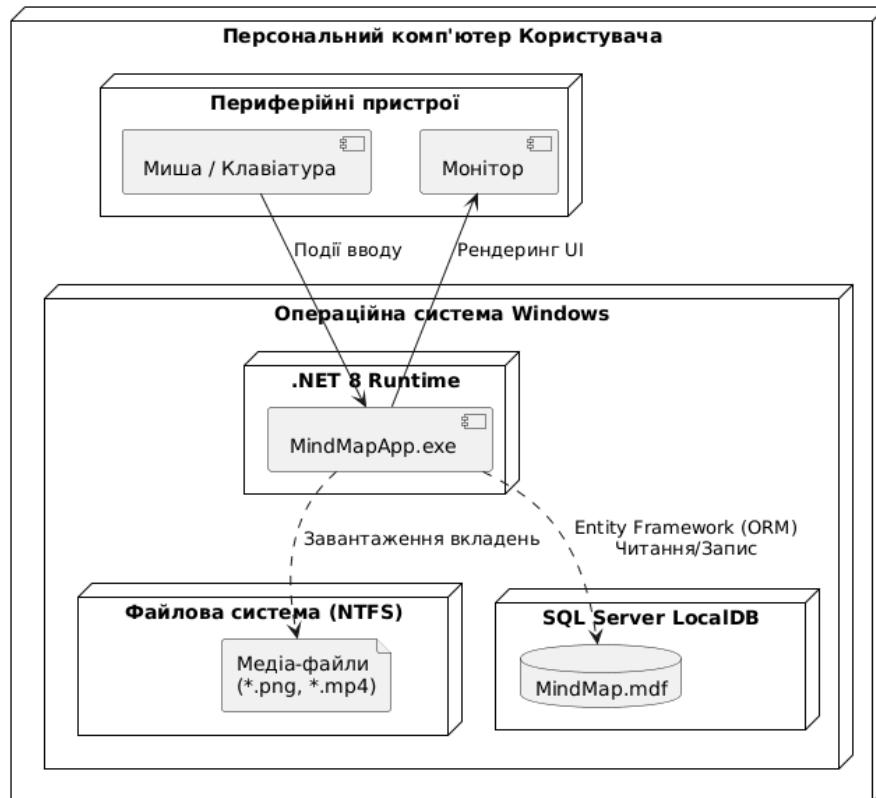


Рис. 1. Діаграма розгортання системи

MindMapApp має архітектуру що реалізована як клієнтський настільний додаток (Desktop Application) для робочої станції користувача.

Функціонування системи відбувається в операційному середовищі Windows на базі платформи .NET 8 Runtime. Взаємодія з користувачем забезпечується через стандартні периферійні пристрої: ввід команд та графічні маніпуляції з елементами карти здійснюються за допомогою миші та клавіатури, для візуалізація інтерфейсу виводиться на монітор. Архітектура зберігання даних побудована за гібридним принципом, де структурована інформація про вузли та зв'язки обробляється через ORM Entity Framework і зберігається у локальній базі даних SQL Server LocalDB, а медіа-вкладення через більший розмір зберігаються і зчитуються безпосередньо з файлової системи робочої станції Користувача, для забезпечення швидкодії системи.

Діаграма компонентів

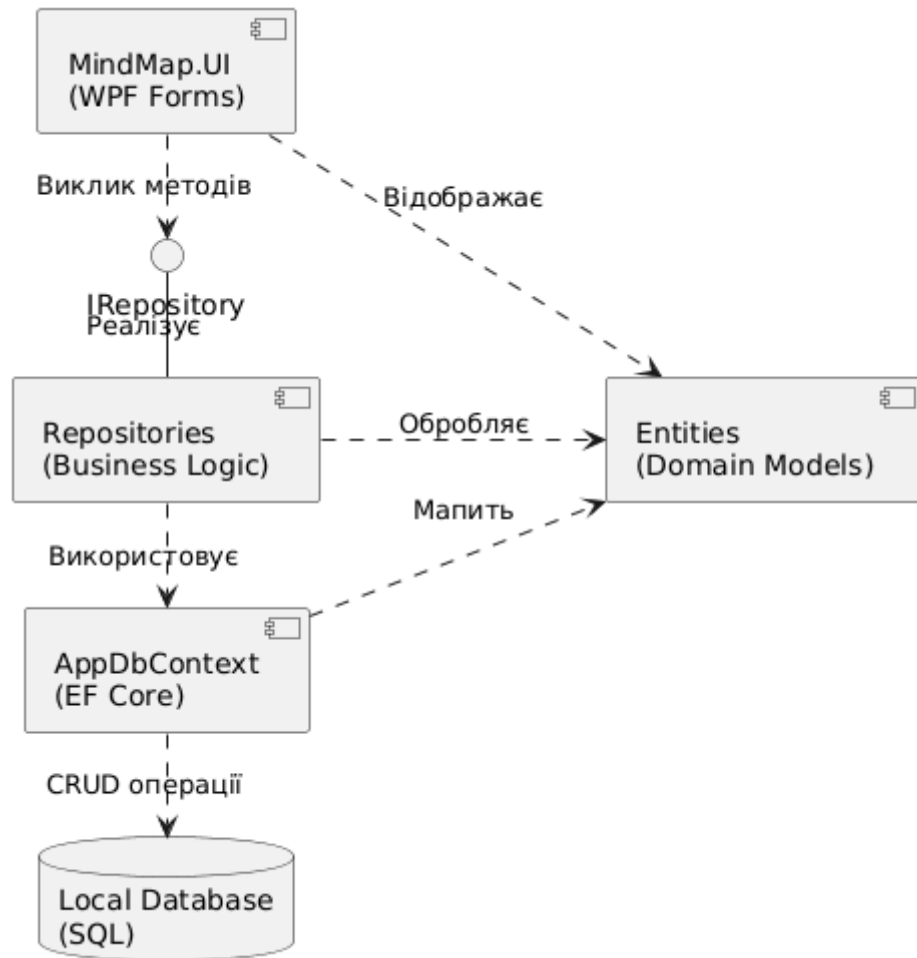


Рис. 2. Діаграма компонентів

На діаграмі компонентів зображено тришарову модульну структуру системи. Верхній рівень займає Шар представлення (UI), реалізований на технології WPF (компоненти MainWindow, EditNodeWindow), і потрібен для взаємодії з користувачем та візуалізації графічних елементів карти. Цей шар не звертається до бази даних, а делегує операції Шару репозиторіїв, взаємодіючи з ним через абстракцію інтерфейсів (IRepository). Репозиторії (MindMapRepository) інкапсулюють бізнес-логіку збереження та пошуку даних, використовуючи компонент Контексту даних (AppDbContext) на базі Entity Framework Core. Контекст даних виступає мостом між об'єктним кодом та реляційною базою даних, транслюючи запити у SQL-команди до фізичного файлу БД (MindMap.mdf). Всі рівні системи об'єднані спільним використанням компонента Об'єктної моделі (Entities), який містить класи-сутності (MindMap, Node), що забезпечує цілісність типів даних у всьому додатку.

Діаграми послідовностей

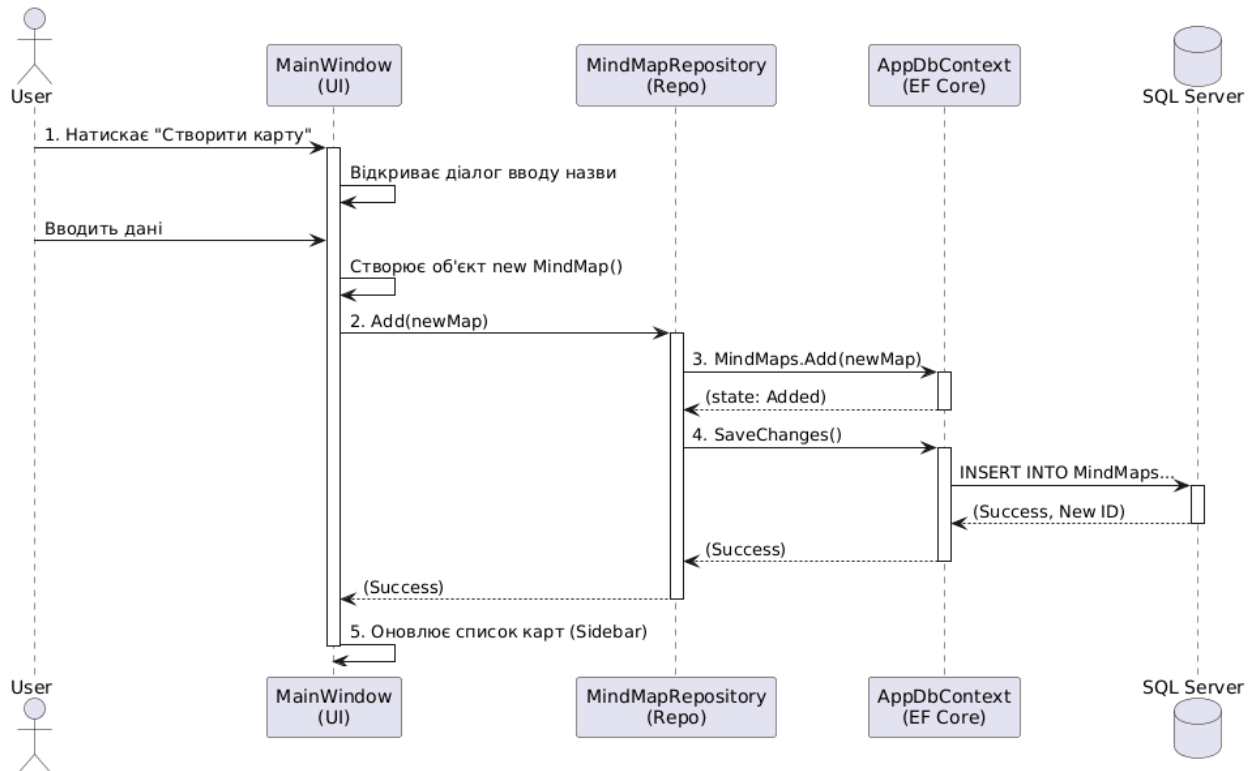


Рис. 3. Діаграма послідовності «Створення нової карти»

Процес починається на рівні інтерфейсу користувача (MainWindow), після введення назви карти створюється новий екземпляр класу. Далі об'єкт передається у шар бізнес-логіки через виклик методу репозиторію, Репозиторій реєструє нову сутність у контексті Entity Framework, переводячи її у стан Added, після чого виклик методу SaveChanges() ініціює транзакцію. Entity Framework генерує відповідний SQL-запит INSERT, виконує його на сервері MS SQL LocalDB та повертає згенерований ідентифікатор нової карти. Після успішного завершення транзакції шар інтерфейсу оновлює список карт у бічній панелі, відображаючи новостворений елемент.

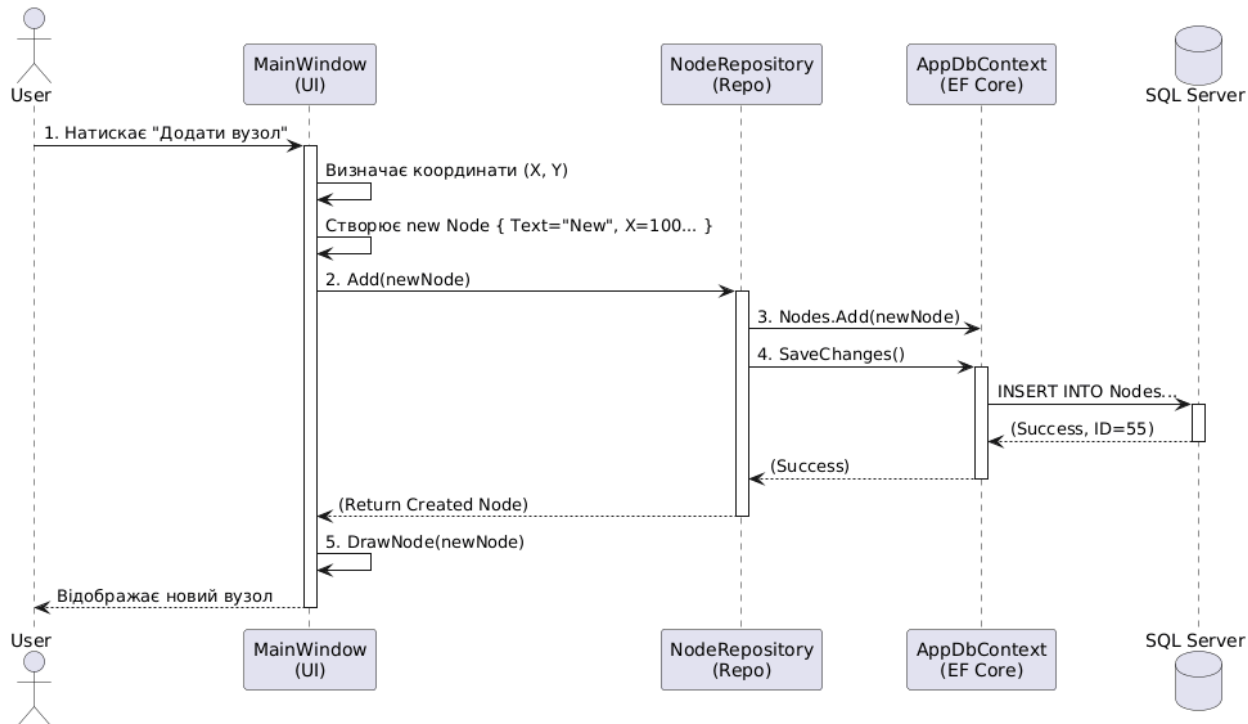


Рис. 4. Діаграма послідовності «Створення нового вузла»

На рівні головного вікна спочатку обчислюються координати розміщення елемента та ініціалізується об'єкт Node із початковими властивостями. Створений об'єкт передається у NodeRepository для збереження, при цьому метод додавання повертає управління лише після того, як база даних присвоїть вузлу унікальний первинний ключ. Після успішного підтвердження транзакції від БД шар представлення викликає внутрішній метод DrawNode, який додає відповідний елемент на компонент Canvas робочої області.

Код програми

MindMapRepository

```

using MindMapApp.Entities;
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using System.Linq;

namespace MindMapApp.Repositories
{
    public class MindMapRepository : IRepository<MindMap>
    {

```



```
private readonly AppDbContext _context;
```

```
public MindMapRepository()  
{  
    _context = new AppDbContext();  
}
```

```
public void Add(MindMap entity)  
{  
    _context.MindMaps.Add(entity);  
    _context.SaveChanges(); // запит в БД  
}
```

```
public void Delete(int id)  
{  
    var item = _context.MindMaps.Find(id);  
    if (item != null)  
    {  
        _context.MindMaps.Remove(item);  
        _context.SaveChanges();  
    }  
}
```

```
public IEnumerable<MindMap> GetAll()  
{  
    return _context.MindMaps.Include(m => m.Nodes).ToList();  
}
```

```
public MindMap GetById(int id)
```

```

        {
            return _context.MindMaps.Include(m => m.Nodes).Include(m =>
m.Connections).FirstOrDefault(m => m.Id == id);
        }

        public void Update(MindMap entity)
        {
            _context.Entry(entity).State = EntityState.Modified;
            _context.SaveChanges();
        }
    }
}

```

AppDbContext

```

using Microsoft.EntityFrameworkCore;
using MindMapApp.Entities;
using System.Collections.Generic;

namespace MindMapApp.Entities
{
    public class AppDbContext : DbContext
    {
        public DbSet<MindMap> MindMaps { get; set; }
        public DbSet<Node> Nodes { get; set; }
        public DbSet<Connection> Connections { get; set; }
        public DbSet<Attachment> Attachments { get; set; }
        public DbSet<Region> Regions { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)

```

```

    {
optionsBuilder.UseSqlServer("Server=(localdb)\mssqllocaldb;Database=Mind
MapDb;Trusted_Connection=True;");
    }
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Connection>()
        .HasOne(c => c.FromNode)
        .WithMany()
        .HasForeignKey(c => c.FromNodeId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Connection>()
        .HasOne(c => c.ToNode)
        .WithMany()
        .HasForeignKey(c => c.ToNodeId)
        .OnDelete(DeleteBehavior.Restrict);
}
}
}

```

MainWindow.xaml (розмітка UI)

```

<Window x:Class="MindMapApp.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="MindMap App" Height="600" Width="900"
    WindowStartupLocation="CenterScreen">

    <Grid>

```

```

<Grid.ColumnDefinitions>
    <ColumnDefinition Width="250"/>
    <ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

<Grid Grid.Column="0" Background="#F0F0F0">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

    <TextBlock Text="Мої карти" FontSize="18" FontWeight="Bold"
Margin="10"/>

    <ListBox x:Name="MapsListBox" Grid.Row="1" Margin="5"
SelectionChanged="MapsListBox_SelectionChanged">
        <ListBox.ItemTemplate>
            <DataTemplate>
                <StackPanel Margin="0,5">
                    <TextBlock Text="{Binding Title}" FontWeight="Bold"/>
                    <TextBlock Text="{Binding CreatedAt, StringFormat=d}"
FontSize="10" Foreground="Gray"/>
                </StackPanel>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>

    <StackPanel Grid.Row="2" Margin="10">

```

```

        <TextBlock Text="Нова карта:" FontSize="12"
Foreground="Gray"/>

        <TextBox x:Name="NewMapTitleBox" Height="25"
Margin="0,0,0,5"/>

        <Button Content="Створити карту" Click="CreateMap_Click"
Height="30" Background="#DDDDDD"/>

    </StackPanel>

</Grid>

<Border Grid.Column="1" BorderBrush="Gray"
BorderThickness="1,0,0,0">

    <Grid>

        <Grid.RowDefinitions>

            <RowDefinition Height="Auto"/>

            <RowDefinition Height="*/>

        </Grid.RowDefinitions>

        <Border Background="#EFEFEF" Padding="5">

            <StackPanel Orientation="Horizontal">

                <TextBlock x:Name="CurrentMapLabel" Text="Оберіть карту
зі списку" VerticalAlignment="Center" FontWeight="Bold"
Margin="0,0,20,0"/>

                <Button Content="Додати Вузол" Click="AddNode_Click"
Padding="5,2" IsEnabled="False" Name="BtnAddNode"/>

            </StackPanel>

        </Border>

        <Canvas x:Name="DrawingCanvas" Grid.Row="1"
Background="White"
MouseLeftButtonDown="Canvas_MouseLeftButtonDown"/>

    </Grid>

</Border>

```

```
</Grid>
```

```
</Window>
```

MainWindow.xaml.cs (логіка роботи)

```
using System;
```

```
using System.Linq;
```

```
using System.Windows;
```

```
using System.Windows.Controls;
```

```
using System.Windows.Input;
```

```
using System.Windows.Media;
```

```
using System.Windows.Shapes;
```

```
using MindMapApp.Entities;
```

```
using MindMapApp.Repositories;
```

```
namespace MindMapApp
```

```
{
```

```
    public partial class MainWindow : Window
```

```
    {
```

```
        private readonly MindMapRepository _repository;
```

```
        private MindMap _currentMap;
```

```
        public MainWindow()
```

```
        {
```

```
            try
```

```
            {
```

```
                InitializeComponent();
```

```
                _repository = new MindMapRepository();
```

```
                LoadMapsList();
```

```
            }
```

```
            catch (Exception ex)
```

```

        {
            MessageBox.Show($"Критична помилка  
запуску:\n{ex.Message}\n\nДеталі:\n{ex.InnerException?.Message}");
        }
    }

    private void LoadMapsList()
    {
        var maps = _repository.GetAll();
        MapsListBox.ItemsSource = maps;
    }

    private void CreateMap_Click(object sender, RoutedEventArgs e)
    {
        if (string.IsNullOrEmpty(NewMapTitleBox.Text)) return;

        var newMap = new MindMap { Title = NewMapTitleBox.Text };
        _repository.Add(newMap);

        NewMapTitleBox.Text = "";
        LoadMapsList();
    }

    private void MapsListBox_SelectionChanged(object sender,
    SelectionChangedEventArgs e)
    {
        if (MapsListBox.SelectedItem is MindMap selectedMap)
        {
            _currentMap = _repository.GetById(selectedMap.Id);

            CurrentMapLabel.Text = _currentMap.Title;
            BtnAddNode.IsEnabled = true;
        }
    }

```

```

        DrawCurrentMap();
    }
}

private void AddNode_Click(object sender, RoutedEventArgs e)
{
    if (_currentMap == null) return;

    var newNode = new Node
    {
        Text = "Новий вузол",
        PosX = 100,
        PosY = 100,
        Color = "#ADD8E6",
        MindMapId = _currentMap.Id
    };
    _currentMap.Nodes.Add(newNode);
    _repository.Update(_currentMap);

    DrawCurrentMap();
}

private void DrawCurrentMap()
{
    DrawingCanvas.Children.Clear();

    if (_currentMap == null || _currentMap.Nodes == null) return;

    foreach (var node in _currentMap.Nodes)

```



```

{
    var ellipse = new Ellipse
    {
        Width = 80,
        Height = 40,
        Fill = (SolidColorBrush)new
BrushConverter().ConvertFrom(node.Color),
        Stroke = Brushes.Black,
        StrokeThickness = 1
    };

    var textBlock = new TextBlock
    {
        Text = node.Text,
        HorizontalAlignment = HorizontalAlignment.Center,
        VerticalAlignment = VerticalAlignment.Center,
        TextTrimming = TextTrimming.CharacterEllipsis,
        MaxWidth = 70
    };

    var grid = new Grid { Width = 80, Height = 40, Cursor =
Cursors.Hand };
    grid.Children.Add(ellipse);
    grid.Children.Add(textBlock);

    Canvas.SetLeft(grid, node.PosX);
    Canvas.SetTop(grid, node.PosY);
    grid.Tag = node;

```

```

        grid.MouseLeftButtonDown += Node_Clicked;

        DrawingCanvas.Children.Add(grid);
    }
}

private void Node_Clicked(object sender, MouseButtonEventArgs e)
{
    if (e.ClickCount == 2)
    {
        var grid = sender as Grid;
        var node = grid.Tag as Node;

        if (node != null)
        {
            var editWindow = new EditNodeWindow(node);
            editWindow.Owner = this; //вікно редагування поверх головного

            if (editWindow.ShowDialog() == true)
            {
                _repository.Update(_currentMap);
                DrawCurrentMap();
            }
        }
    }
}

private void Canvas_MouseLeftButtonDown(object sender,
MouseButtonEventArgs e)
{
}

```

```
}  
}
```

EditNodeWindow.xaml (розмітка UI)

```
<Window x:Class="MindMapApp.EditNodeWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Редагування вузла" Height="250" Width="300"  
    WindowStartupLocation="CenterOwner" ResizeMode="NoResize">  
  
    <StackPanel Margin="20">  
        <TextBlock Text="Текст вузла:"/>  
        <TextBox x:Name="NodeTextBox" Margin="0,5,0,15" Height="25"/>  
  
        <TextBlock Text="Колір (HEX):"/>  
        <ComboBox x:Name="ColorComboBox" Margin="0,5,0,20"  
SelectedIndex="0">  
            <ComboBoxItem Content="#ADD8E6" Background="#ADD8E6"  
Tag="Блакитний"/>  
            <ComboBoxItem Content="#90EE90" Background="#90EE90"  
Tag="Зелений"/>  
            <ComboBoxItem Content="#FFB6C1" Background="#FFB6C1"  
Tag="Рожевий"/>  
            <ComboBoxItem Content="#FFE4B5" Background="#FFE4B5"  
Tag="Бежевий"/>  
            <ComboBoxItem Content="#FFFFFF" Background="#FFFFFF"  
Tag="Білий"/>  
        </ComboBox>  
  
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Right">  
            <Button Content="Зберегти" Click="Save_Click" Width="80"  
Margin="0,0,10,0"/>
```

```
        <Button Content="Скасувати" Click="Cancel_Click" Width="80"/>
    </StackPanel>
</StackPanel>
</Window>
```

EditNodeWindow.xaml.cs

```
using System.Windows;
using System.Windows.Controls;
using MindMapApp.Entities;

namespace MindMapApp
{
    public partial class EditNodeWindow : Window
    {
        public Node EditingNode { get; private set; }

        public EditNodeWindow(Node nodeToEdit)
        {
            InitializeComponent();
            EditingNode = nodeToEdit;
            NodeTextBox.Text = EditingNode.Text;

            foreach (ComboBoxItem item in ColorComboBox.Items)
            {
                if (item.Content.ToString() == EditingNode.Color)
                {
                    ColorComboBox.SelectedItem = item;
                    break;
                }
            }
        }
    }
}
```

```

    }

    private void Save_Click(object sender, RoutedEventArgs e)
    {
        EditingNode.Text = NodeTextBox.Text;

        if (ColorComboBox.SelectedItem is ComboBoxItem selectedItem)
        {
            EditingNode.Color = selectedItem.Content.ToString();
        }

        DialogResult = true;
        Close();
    }

    private void Cancel_Click(object sender, RoutedEventArgs e)
    {
        DialogResult = false;
        Close();
    }
}
}
}

```

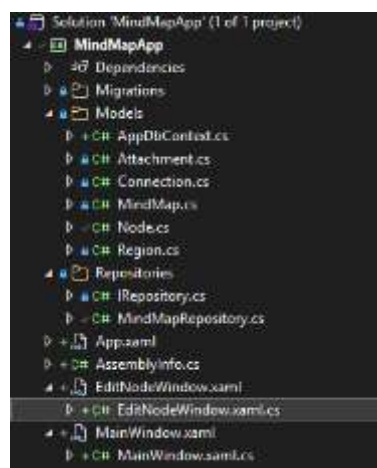


Рис. 5. Структура додатку

Висновок: В ході виконання лабораторної роботи було розроблено базову структуру проекту, виконано підключення бази даних, створено діаграму компонентів, діаграму розгортання і діаграми послідовностей для двох сценаріїв використання для кращого розуміння логіки роботи додатку що полегшить подальшу розробку

Питання до лабораторної роботи

1. Що собою становить діаграма розгортання?
Діаграми розгортання демонструють фізичне розташування системи, показуючи яке фізичне обладнання забезпечує функціонування різних частин системи
2. Які бувають види вузлів на діаграмі розгортання?
Пристрій – фізичне обладнання, наприклад комп'ютер або пристрій пов'язаний із системою. Середовище виконання – програмне забезпечення яке може включати операційну систему або вебсервер
3. Які бувають зв'язки на діаграмі розгортання?
Асоціація – показує шлях обміну даними між вузлами. Залежність – показує що елемент залежить від іншого
4. Які елементи присутні на діаграмі компонентів?
Компоненти – модульні частини системи, інтерфейси – визначають набір операцій які компонент надає або вимагає, порти – точки взаємодії компонента з оточенням, артефакти – фізичні файли що реалізують компонент
5. Що становлять собою зв'язки на діаграмі компонентів?
Це логічні з'єднання, які показують, як компоненти пов'язані. Зазвичай це реалізація інтерфейсу (компонент надає сервіс) або залежність (компонент використовує сервіс іншого компонента).
6. Які бувають види діаграм взаємодії?
Діаграма послідовностей, діаграма комунікації, діаграма часу, діаграма огляду взаємодії
7. Для чого призначена діаграма послідовностей?
Для візуалізації взаємодії об'єктів у часі. Вона показує, у якому порядку об'єкти обмінюються повідомленнями для виконання конкретного сценарію.
8. Які ключові елементи можуть бути на діаграмі послідовностей?
Актори, об'єкти або класи, повідомлення, активності, контрольні структури,
9. Як діаграми послідовностей пов'язані з діаграмами варіантів використання?
Діаграма послідовності деталізує один конкретний варіант використання. Вона показує як технічно реалізується сценарій, описаний у варіанті використання.

10. Як діаграми послідовностей пов'язані з діаграмами класів?

Лінії життя на діаграмі послідовності — це екземпляри Класів з діаграми класів.

Повідомлення, які вони надсилають — це виклики Методів, описаних у цих класах.